

计算机体系结构

新型计算机研究所

陈 衡

西一楼 A 段415室

Email: hengchen@xjtu.edu.cn

QQ群号: 1031141115

第6章 指令级并行的开发 ——软件方法

- ◆ 6.1 基本指令调度和循环展开
- ◆ 6.2 跨越基本块的静态指令调度
- ◆ 6.3 静态多指令流出

6.1 基本指令调度和循环展开

◆ 6.1.1 指令调度的基本方法

- 1、充分开发指令之间存在的并行性，找出不相关的指令序列，让它们在流水线上重叠并行执行。
- 2、增加指令间并行性最简单和最常用的方法
 - 开发循环级并行性—循环的**不同迭代之间存在的并行性**
 - 在把循环展开后，通过重命名和指令调度来开发更多的并行性。
- 3、编译器完成这类指令调度的能力受限于两个特性：
 - 程序固有的指令级并行性；
 - **流水线功能部件的执行延迟。**

6.1.1 指令调度的基本方法

- ◆ 本节中，我们使用的浮点流水线延迟为：

产生结果的指令	使用结果的指令	延迟（时钟周期数）
浮点计算	另一个浮点计算	3
浮点计算	浮点store (S.D)	2
浮点load (L.D)	浮点计算	1
浮点load (L.D)	浮点store (S.D)	0

- ◆ 假设采用第3章的5段整数流水线：
 - 分支的延迟：1个时钟周期。
 - 整数load指令的延迟：1个时钟周期。
 - 整数运算部件是全流水或者重复设置了足够的份数。

指令调度举例

- ◆ 例6.1 对于下面的源代码，转换成MIPS汇编语言，在不进行指令调度和进行指令调度两种情况下，分析其一次循环所需的执行时间。

```
for (i=1000; i>0; i--)  
    x[i] = x[i] + s;
```

- ◆ 解：把该程序翻译成MIPS汇编语言代码：

```
Loop: L.D      F0, 0(R1)           // 取一个向量元数放入F0  
      ADD.D    F4, F0, F2          // 加上在F2中的标量  
      S.D      F4, 0(R1)          // 存结果  
      DADDIU   R1, R1, #-8         // 指针减去8  
      BNE      R1, R2, Loop        // 未处理完的话，继续
```

- ◆ 其中：

- R1：指向向量中的当前元素，初值为最后1个元素的地址
- 8(R2)：指向第一个元素。
- 浮点寄存器F2：用于保存常数s。

指令调度举例

- ◆ 不进行指令调度的情况下，程序的实际执行情况：

标号	指令	时钟周期
Loop:	L.D F0, 0(R1)	1
	(空转)	2
	ADD.D F4, F0, F2	3
	(空转)	4
	(空转)	5
	S.D F4, 0(R1)	6
	DADDIU R1, R1, #-8	7
	(空转)	8
	BNE R1, R2, Loop	9
	(空转)	10

- ◆ 每个元素的操作需要10个时钟周期，
- ◆ 其中5个是空转周期。

指令调度举例

- ◆ 指令调度以后，程序的执行情况如下：
 - 把DADDIU指令调度到了L.D指令和ADD.D指令之间的“空转”拍
 - 把S.D指令放到了分支指令的延迟槽中
 - 对存储器地址偏移量进行调整

标号	指令
Loop:	L.D F0, 0(R1)
	(空转)
	ADD.D F4, F0, F2
	(空转)
	(空转)
	S.D F4, 0(R1)
	DADDIU R1, R1, #-8
	(空转)
	BNE R1, R2, Loop
	(空转)

标号	指令
Loop:	L.D F0, 0(R1)
	DADDIU R1, R1, #-8
	ADD.D F4, F0, F2
	(空转)
	BNE R1, R2, Loop
	S.D F4, 8(R1)

指令调度举例

- ◆ 一个元素的操作时间从10个时钟周期减少到6个，其中5个周期是有指令执行的，1个为空转周期

- ◆ 例子中的**问题**及解决方案

- 只有L.D、ADD.D和S.D这3条指令是有效操作，占用3个时钟周期。
- 而DADDIU、空转和BNE这3个时钟周期都是附加的循环控制开销。
- 要减少这种附加的循环控制开销在程序执行时间中所占的比率，可以使用**循环展开**技术

标号	指令		时钟
Loop:	L.D	F0, 0(R1)	1
	DADDIU	R1, R1, #-8	2
	ADD.D	F4, F0, F2	3
	(空转)		4
	BNE	R1, R2, Loop	5
	S.D	F4, 8(R1)	6

6.1.2 循环展开

◆ 循环展开技术：

- 把循环体代码复制多次并按顺序排列，调整循环结束条件，为指令调度带来更大的空间
- ◆ 例6.2 将上述例子中的循环展开3次得到4个循环体，然后对展开后的指令序列在不调度和调度两种情况下，分析代码的性能。**假定R1的初值为256**，即循环次数为4的倍数。消除冗余的指令，并且不要重复使用寄存器。
- ◆ 解：展开后迭代次数为8，且无需在循环体后面增加补偿代码

- 分配寄存器
- 不重复使用寄存器

寄存器	用途
F2	保存常数
F0、F4	展开后的第1个循环体
F6、F8	展开后的第2个循环体
F10、F12	展开后的第3个循环体
F14、F16	展开后的第4个循环体

循环展开举例——没有调度的代码

标号	指令	时钟
Loop:	L.D F0, 0(R1)	1
	(空转)	2
	ADD.D F4, F0, F2	3
	(空转)	4
	(空转)	5
	S.D F4, 0(R1)	6
	L.D F6, -8(R1)	7
	(空转)	8
	ADD.D F8, F6, F2	9
	(空转)	10
	(空转)	11
	S.D F8, -8(R1)	12
	L.D F10, -16(R1)	13
	(空转)	14

标号	指令	时钟
	ADD.D F12, F10, F2	15
	(空转)	16
	(空转)	17
	S.D F12, -16(R1)	18
	L.D F14, -24(R1)	19
	(空转)	20
	ADD.D F16, F14, F2	21
	(空转)	22
	(空转)	23
	S.D F16, -24(R1)	24
	DADDIU R1, R1, #-32	25
	(空转)	26
	BNE R1, R2, Loop	27
	(空转)	28

循环展开举例—没有调度的代码

◆ 结果分析：

- 这个循环每遍共使用了28个时钟周期。
- 有4个循环体，完成4个元素的操作。
- 平均每个元素使用 $28/4=7$ 个时钟周期
- 原始循环的每个元素需要10个时钟周期。
- 节省的时间：从减少循环控制的开销中获得的。
- 在整个展开后的循环中，实际指令只有14条，其他14个周期都是空转。

◆ 效率并不高

循环展开举例—优化调度的代码

- ◆ 对指令序列进行优化调度，以减少空转周期
- ◆ 结果分析
 - 没有数据相关引起的空转等待。
 - 整个循环仅仅使用了14个时钟周期。
 - 平均每个元素的操作使用 $14/4=3.5$ 个时钟周期
 - 通过循环展开、寄存器重命名和指令调度，可以有效地开发出指令级并行。

标号	指令		时钟
Loop:	L.D	F0, 0(R1)	1
	L.D	F6, -8(R1)	2
	L.D	F10, -16(R1)	3
	L.D	F14, -24(R1)	4
	ADD.D	F4, F0, F2	5
	ADD.D	F8, F6, F2	6
	ADD.D	F12, F10, F2	7
	ADD.D	F16, F14, F2	8
	S.D	F4, 0(R1)	9
	S.D	F8, -8(R1)	10
	DADDIU	R1, R1, #-32	11
	S.D	F12, 16(R1)	12
	BNE	R1, R2, Loop	13
	S.D	F16, 8(R1)	14

循环展开和指令调度注意事项

- ◆ **保证正确性**。在循环展开和调度过程中尤其要注意两个地方的正确性：**循环控制**，**操作数偏移量的修改**。
- ◆ **注意有效性**。只有能够找到不同循环体之间的无关性，才能有效地使用循环展开。
- ◆ **使用不同的寄存器**。否则可能导致新的冲突
- ◆ **删除多余的测试指令和分支指令**，并对循环结束代码和新的循环体代码进行相应的修正。
- ◆ **注意对存储器数据的相关性分析**
 - 例如：对于load指令和store指令，如果它们在不同的循环迭代中访问的存储器地址是不同的，它们就是相互独立的，可以相互对调。
- ◆ **注意新的相关性**。由于原循环不同次的迭代在展开后都到了同一次循环体中，因此可能带来新的相关性。

6.2 跨越基本块的静态指令调度

◆ 6.2.1 全局指令调度

◆ 目标：

- 在保持原有数据相关和控制相关不变的前提下，尽可能地**缩短包含分支结构的代码段的总执行时间**。
 - 单流出处理器—减少指令数
 - 多流出处理器—缩短关键路径长度
- **关键路径**：根据指令间的相互关系构成的数据流图中延迟最长的一条路径。

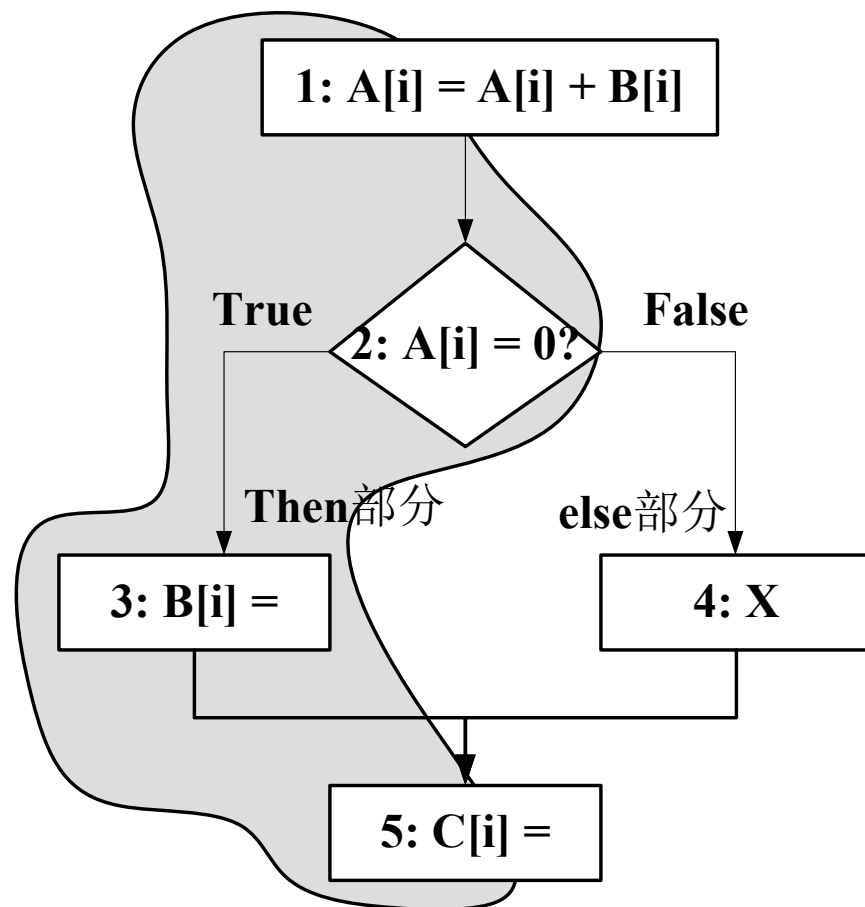
◆ 基本思想：

- 在循环体内的多个基本块间移动指令，扩大那些执行频率较高的基本块的体积。

6.2.1 全局指令调度

◆ 分析:

- 由于分支条件为true(转移)的概率大, 全局指令调度时会把语句1、2、3、5合并为一个更大的基本块。
- 如何保证分支条件为false时结果依然正确?
- 为了完成合并, 如何将语句3和5调度到语句2之前? 语句3靠猜测执行, 而语句5肯定会被执行。



一个分支结构的代码段

6.2.1 全局指令调度

◆ 编译器的做法：

- 将上图中的代码转换为下面的MIPS汇编指令

```
LD      R4, 0(R1)           // 取A
LD      R5, 0(R2)           // 取B
DADDU   R4, R4, R5          // A=A+B
SD      0(R1), R4           // 存A
BEQZ    R4, thenpart        // A=0则转移
X                               // 代码段X, 基本块elsepart
J      join
```

```
thenpart:                    // 基本块thenpart
```

```
SD ..., 0(R2)               // 指令I1, 对应语句3
```

```
join:
```

```
SD ..., 0(R3)               // 指令I2, 对应语句5
```


6.2.1 全局指令调度

◆ 调度指令I1

- 直接将I1移到BEQZ前。若基本块elsepart中有语句使用了变量B的值，是否会产生错误结果？
- 向基本块elsepart中增加补偿代码，使用临时变量T，将指令中使用源操作数B修改为T
- 在编译实现时较复杂，同时补偿代码的执行会带来额外时间开销

◆ 调度指令I2

- 将I2移动到基本块thenpart中，同时复制到elsepart中。
- 若不影响执行结果，将I2调度到BEQZ前，同时删除elsepart中的副本。

6.2.1 全局指令调度

- ◆ 全局指令调度是一个很复杂的问题
 - 选择的依据是：调度指令后是否一定会带来性能提升？
- ◆ 以I1的调度为例：
 - 需要确定分支中基本块thenpart和elsepart的执行频率各是多少？若thenpart块执行频繁，则会带来性能提升
 - 在分支语句前完成I1所需的开销是多大？若分支语句前有“空转”周期，则所需开销为0
 - 调度I1是否能够缩短thenpart块的执行时间？如果I1是该块关键路径的第1条语句，则会减少调度开销
 - I1是否是最佳的被调度对象？
 - 是否需要向elsepart块中增加补偿代码，补偿代码开销如何？怎样生成补偿代码？

6.2.2 踪迹调度

◆ 踪迹（trace）：

- 程序执行的指令序列，通常由一个或多个基本块组成，trace内可以有分支，但一定不能包含循环。

◆ 踪迹调度（trace scheduling）

- 踪迹调度会优化执行频率高的trace，减少其执行开销。由于需要添加补偿代码以确保正确性，那些执行频率较低的trace的开销反而会有所增加。
- 踪迹调度非常适合多流出处理器。

◆ 踪迹调度的步骤，分为两步：

- 踪迹选择
- 踪迹压缩

6.2.2 踪迹调度

◆ 踪迹选择

- 从程序的控制流图中选择执行频率较高的路径，每条路径就是一条trace。
- 适合于处理转移成功与失败概率相差较大的情况
- 循环结构：循环展开，基于循环体内的执行频率会远远高于循环体外其他基本块的执行频率
- 分支结构：根据静态分支的预测结果或根据典型输入集下的运行统计信息进行处理。

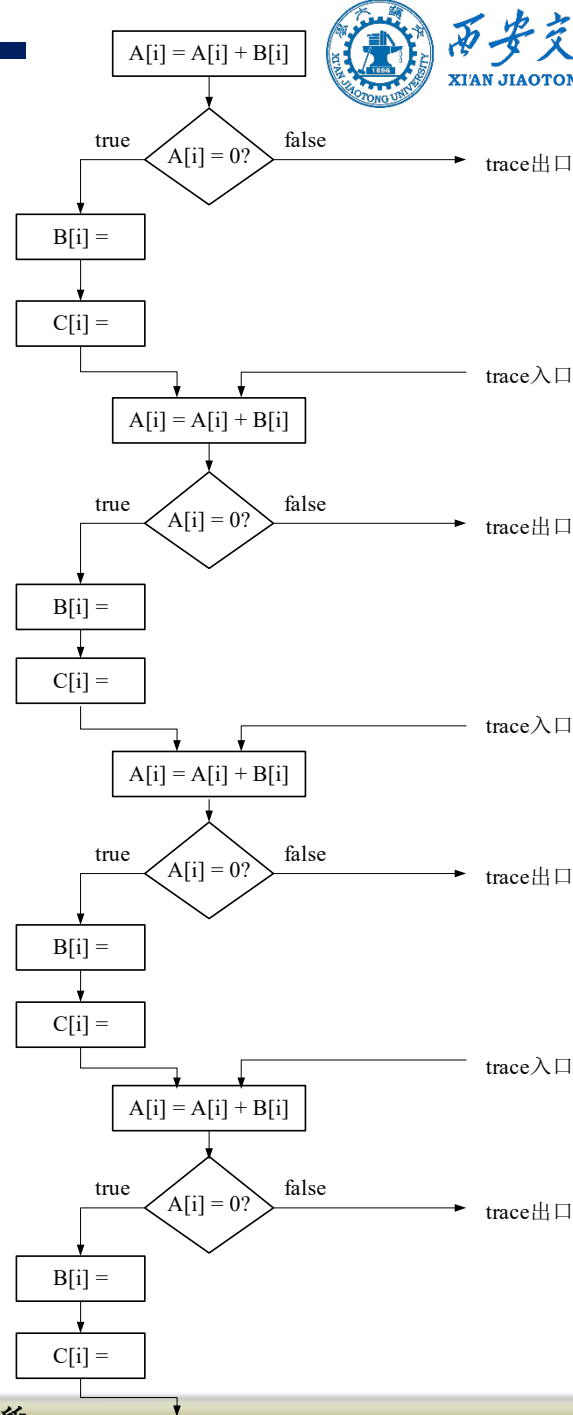
◆ 举例

- 以图6.1中的代码为例
- 假设它是某循环的循环体，阴影部分（thenpart）是执行频率高的路径

6.2.2 踪迹调度

◆ 踪迹选择实例分析

- 将循环展开4次并把阴影部分 (thenpart) 拼接在一起就可以得到一条trace;
- 一条trace可以有多个入口和多个出口。
- 入口指的是控制流进入trace后执行的第1条语句, 这里是4条 “ $A[i]=A[i]+B[i]$ ” 语句。
- 出口是控制流离开trace前执行的最后1条语句, 这里是4条当分支条件为false, 以及最后1条语句



6.2.2 踪迹调度

◆ 踪迹压缩

- 对已生成的trace进行指令调度和优化，重新安排trace内指令的执行顺序，尽可能地缩短其执行时间；
- 跨越trace内部的入口或出口调度指令时必须非常小心，有可能改变原有的控制相关和数据相关，从而造成执行错误。通常需要向trace外该入口的前驱基本块或该出口的后继基本块增加**补偿代码**。

◆ 踪迹调度的性能特点

- 踪迹调度能够提升性能的最根本原因在于选出的trace都是执行频率很高的路径，减少它们的执行开销有助于缩短程序的总执行时间。
- 对于某些应用，补偿代码引起的开销很有可能降低踪迹调度的优化效果。
- 踪迹调度会大大增加编译器的实现复杂度。

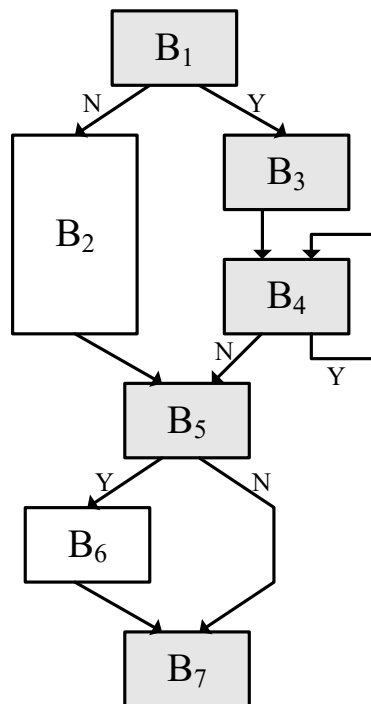
6.2.2 踪迹调度

◆ 踪迹压缩实例分析

- 假设控制流离开基本块B1和B4后发生转移（分支条件Y）的概率很大，离开B5后发生转移的概率很小
- 三条trace: B1-B3、B4以及B5-B7

以traceB₁-B₃为例:

- 指令“ $y = x - y$ ”被从B₁调度到B₃中，跨越了trace的一个出口；
- 需要向块B₂中增加补偿代码，即将指令“ $y = x - y$ ”复制到B₂的第一条指令之前。



(a) 控制流程图

B₁: $x = x + 1$
 $y = x - y$
 if $x < 5$ goto B₃

B₂: $z = x * z$
 $y = y + 1$
 goto B₅

B₃: $y = 2 * y$
 $x = x - 2$

压缩前

B₁: $x = x + 1$
 if $x < 5$ goto B₃

B₂: $y = x - y$
 $z = x * z$
 $y = y + 1$
 goto B₅

B₃: $y = x - y$
 $y = 2 * y$
 $x = x - 2$

压缩后

(b) 踪迹压缩前后的基本块B₁、B₂、B₃

6.2.3 超块调度

◆ 问题

- 在踪迹调度中，如果trace入口或出口位于trace内部，编译器生成补偿代码的难度将大大增加，而且编译器很难评估这些补偿代码究竟会带来多少性能损失。

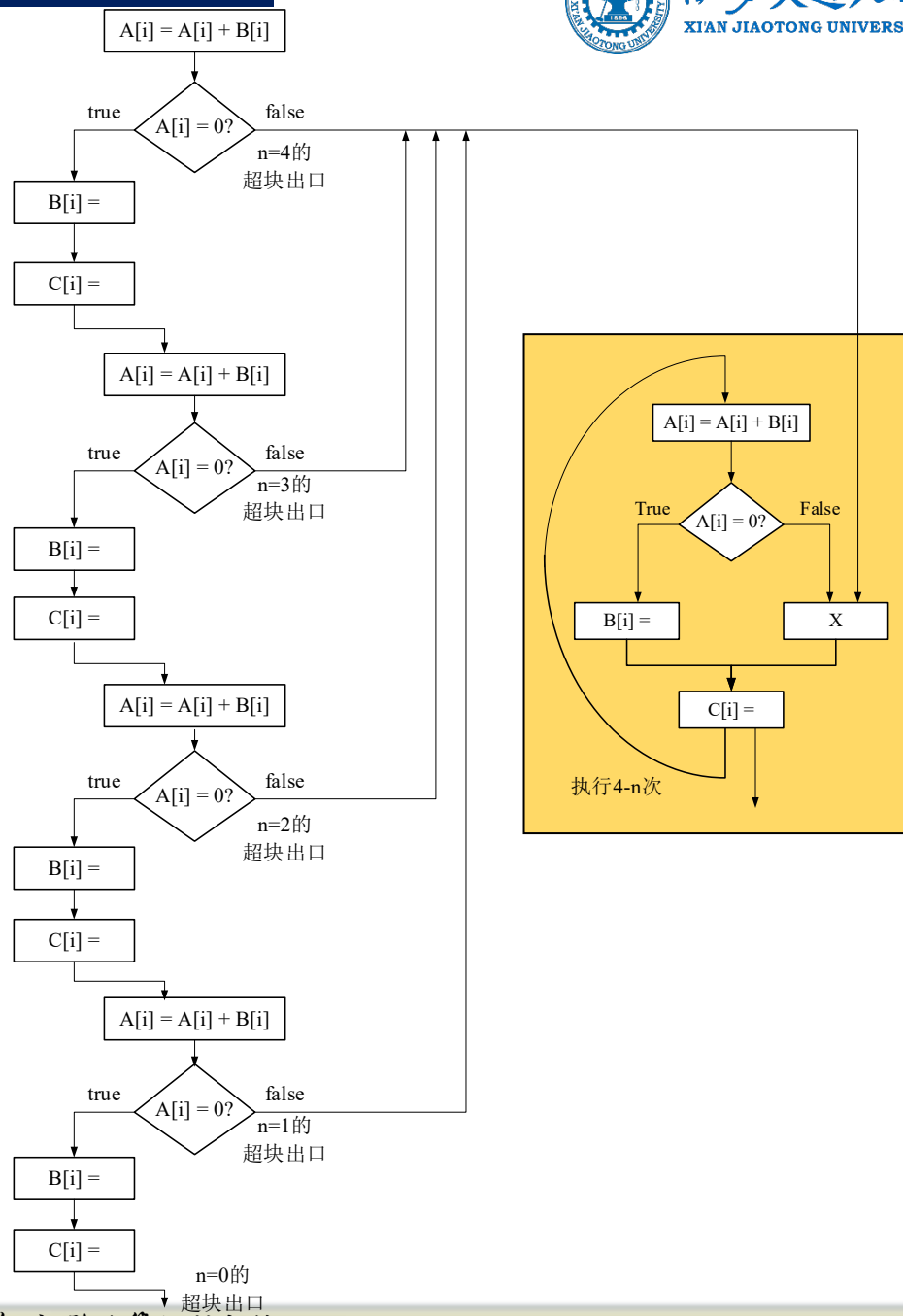
◆ 超块（superblock）

- 为解决上述踪迹调度问题，通过对trace拓扑结构进行约束，使其变成只能拥有一个入口，但可以拥有多个出口的结构，把这种新结构叫超块。
- 超块的构造过程与trace相似，但怎样确保只有一个入口？

6.2.3 超块调度

◆ 实例分析

- 将左边的循环展开4次并把阴影部分(执行频率高)拼接在一起就可以得到一个超块;
- 超块有1个入口和5个出口($n=4/3/2/1/0$)。
- 除了 $n=0$ 外, 从其他4个出口退出超块后, 还需要继续完成余下的 n 次叠代(黄色部分)。



6.2.3 超块调度

◆ 尾复制

- 由于被复制的代码段总是做为退出超块后必须执行的补偿代码，把这种技术称为**尾复制**。

◆ 超块调度的性能特点

- 尾复制技术简化了补偿代码的生成过程，并降低了指令调度的复杂度。
- 超块结构目标代码的体积也大大增加。
- 补偿代码的生成使得编译过程更加复杂，而且由于无法准确评估由补偿代码引起的时间开销，这限制方法超块调度的应用范围。

6.3 静态多指令流出

◆ VLIW vs. 超标量

- 在动态调度的超标量处理器中，相关检测和指令调度基本都由**硬件**完成。
 - 在静态调度的超标量处理器中，部分相关检测和指令调度工作交由**编译器**完成。
 - 在VLIW处理器中，相关检测和指令调度工作全部由编译器完成，它需要更“智能”的编译器。
- ◆ 仍以例6.1为例来说明VLIW和超标量处理器如何进行循环展开和指令调度？

Loop: L.D	F0, 0(R1)	// 取一个向量元数放入F0
ADD.D	F4, F0, F2	// 加上在F2中的标量
S.D	F4, 0(R1)	// 存结果
DADDIU	R1, R1, #-8	// 指针减去8
BNE	R1, R2, Loop	// 继续

6.3.1 静态多指令流出：VLIW技术

◆ 实例分析

- 例6.3 假设某VLIW处理器每个时钟周期可以同时流出5个操作，包括2个访存操作，2个浮点操作以及1个整数或分支操作。将例6.1中的代码循环展开，并调度到该VLIW处理器上执行。循环展开次数不定，但至少能够保证消除所有流水线“空转”周期，同时不考虑分支延迟

访存操作1	访存操作2	浮点操作1	浮点操作2	整数分支操作
L.D F0, 0(R1)	L.D F6, -8(R1)	nop	nop	nop
L.D F10, -16(R1)	L.D F14, -24(R1)	nop	nop	nop
L.D F18, -32(R1)	L.D F22, -40(R1)	ADD.D F4, F0, F2	ADD.D F8, F6, F2	nop
L.D F26, -48(R1)	nop	ADD.D F12, F10, F2	ADD.D F16, F14, F2	nop
nop	nop	ADD.D F20, F18, F2	ADD.D F24, F22, F2	nop
S.D 0(R1), F4	S.D -8(R1), F8	ADD.D F28, F26, F2	nop	nop
S.D -16(R1), F12	S.D -24(R1), F16	nop	nop	nop
S.D -32(R1), F20	S.D -40(R1), F24	nop	nop	DADDUI R1, R1, #56
S.D 8(R1), F28	nop	nop	nop	BNE R1, R2, Loop

6.3.1 静态多指令流出：VLIW技术

◆ 实例分析

- 解：循环被展开7次，经调度后可以消除所有流水线“空转”。在不考虑分支延迟的情况下，每执行一个叠代需要9个时钟周期，计算出7个结果，即平均每得到一个结果需要1.29个周期。

◆ VLIW的不足：

- VLIW目标代码编码效率低
 - 本例中，编码效率仅比50%略高一些
 - 为消除流水线“空转”需要增加循环展开的次数
 - 很难从应用程序中找到足够多的并行指令填满VLIW指令中的每一个slot
- 所需要的寄存器数量也大大增加

6.3.2 静态多指令流出：超标量处理器

- ◆ 超标量处理器如何进行循环展开和指令调度？
- ◆ 例 下面是例6.1使用的循环程序段，对其进行循环展开，并在超标量流水线上进行调度。

Loop: L.D	F0, 0(R1)	// 取一个向量元数放入F0
ADD.D	F4, F0, F2	// 加上在F2中的标量
S.D	F4, 0(R1)	// 存结果
DADDIU	R1, R1, #-8	// 指针减去8
BNE	R1, R2, Loop	// 继续

静态超标量处理机中的循环展开举例

- ◆ 要达到无延迟的指令调度，须将循环展开5遍并调度
- ◆ 可使S.D指令不会停顿
 - 使用ADD.D的结果

标号	指令
Loop:	L.D F0, 0(R1)
	L.D F6, -8(R1)
	L.D F10, -16(R1)
	L.D F14, -24(R1)
	L.D F18, -32(R1)

标号	指令
	ADD.D F4, F0, F2
	ADD.D F8, F6, F2
	ADD.D F12, F10, F2
	ADD.D F16, F14, F2
	ADD.D F20, F18, F2
	S.D F4, 0(R1)
	S.D F8, -8(R1)
	S.D F12, -16(R1)
	DADDIU R1, R1, #-40
	S.D F16, 16(R1)
	BNE R1, R2, Loop
	S.D F20, 8(R1)



针对超标量进行调度后的指令序列

	整数指令		浮点指令	时钟
Loop:	L.D	F0, 0(R1)		1
	L.D	F6, -8(R1)		2
	L.D	F10, -16(R1)	ADD.D F4, F0, F2	3
	L.D	F14, -24(R1)	ADD.D F8, F6, F2	4
	L.D	F18, -32(R1)	ADD.D F12, F10, F2	5
	S.D	F4, 0(R1)	ADD.D F16, F14, F2	6
	S.D	F8, -8(R1)	ADD.D F20, F18, F2	7
	S.D	F12, -16(R1)		8
	DADDIU	R1, R1, #-40		9
	S.D	F16, 16(R1)		10
	BNE	R1, R2, Loop		11
	S.D	F20, 8(R1)		12

静态超标量处理机中的循环展开

- ◆ 每次循环需12个时钟周期，即计算每个结果需要2.4个时钟周期。（每次循环计算5个结果）
- ◆ 在普通的MIPS流水线上，没有调度的代码迭代一次为10个时钟周期，调度后为6个时钟周期，展开4次并调度后每个迭代为3.5个时钟周期。
- ◆ 与之相比，超标量流水线的性能提高分别为：4.2倍、2.5倍、1.5倍
- ◆ 在这个例子中可以看到，超标量MIPS流水线的性能主要受限于：**整数计算和浮点计算之间的平衡问题**
- ◆ 本例中没有足够的浮点指令来使两路流水线都达到饱和。